

```

    if (btn) {
      btn.classList.add('is-leaving')
    }
    setTimeout(() => {
      router.push(route)
    }, NAVIGATION_DELAY)
  }

function startGame(event) {
  navigateTo('/difficulty', event)
}

function goToExplore(event) {
  navigateTo('/explore', event)
}

onMounted(() => {
  requestAnimationFrame(() => {
    isReady.value = true
  })
})
</script>

<template>
  <div class="home-page">
    <!-- 动态背景 -->
    <div class="bg-decorations">
      <span
        v-for="(sym, i) in floatingSymbols"
        :key="i"
        class="float-item"
        :style="{ '--delay: `${i * 1.5}s`, '--left: `${10 + i * 15}%`, '--top: `${20 + (i % 3) * 20}%` }"
      >
        {{ sym }}
      </span>
    </div>

    <div class="home-shell" :class="{ 'is-ready': isReady }">
      <header class="logo-area">
        <div class="logo-circle">
          <Sparkles :size="32" class="logo-sparkle" />
          <div class="logo-inner">
            <span class="logo-plus">+</span>
            <span class="logo-minus">-</span>
          </div>
        </div>
      </header>

      <section class="hero-panel">
        <div class="hero-topline">

```

```
<span class="mission-chip">
  <Sparkles :size="14" />
  <span>启蒙数学</span>
</span>
<span class="mission-note">{{ TOTAL_LEVELS }}关卡</span>
</div>

<div class="hero-copy">
  <h1 class="title">算一算</h1>
  <p class="subtitle">有趣又好玩的数学游戏。</p>
</div>

<div v-if="completedCount > 0 || (stats.totalCorrect || 0) > 0" class="stats-grid">
  <div class="stat-item">
    <span class="stat-value">{{ completedCount }}</span>
    <span class="stat-label">已通关</span>
  </div>
  <div class="stat-divider"></div>
  <div class="stat-item">
    <span class="stat-value">{{ stats.totalCorrect || 0 }}</span>
    <span class="stat-label">累计答对</span>
  </div>
</div>

<div v-else class="welcome-hint">
  <Sparkles :size="20" class="welcome-sparkle" />
  <span>开始你的第一关吧! </span>
</div>
</section>

<section class="action-panel">
  <button class="btn-main" data-testid="start-challenge-btn" @click="startGame($event)">
    <span class="btn-main-icon">
      <Play :size="24" />
    </span>
    <span class="btn-text">开始挑战</span>
    <ArrowRight :size="20" class="btn-arrow" />
  </button>
  <button class="btn-main btn-explore" data-testid="explore-btn" @click="goToExplore($event)">
    <span class="btn-main-icon">
      <Binary :size="24" />
    </span>
    <span class="btn-text">数字探索</span>
    <ArrowRight :size="20" class="btn-arrow" />
  </button>
</section>
</div>
</div>
</template>

<style scoped>
```

```
.home-page {  
  min-height: 100vh;  
  min-height: 100dvh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 24px 18px;  
  background: radial-gradient(circle at top left, #f8faff 0%, #eef4ff 100%);  
  position: relative;  
  overflow: hidden;  
}
```

```
/* 动态背景装饰 */
```

```
.bg-decorations {  
  position: absolute;  
  inset: 0;  
  pointer-events: none;  
  z-index: 0;  
}
```

```
.float-item {  
  position: absolute;  
  left: var(--left);  
  top: var(--top);  
  font-size: 28px;  
  font-weight: 900;  
  color: rgba(49, 120, 246, 0.04);  
  animation: float 6s ease-in-out infinite;  
  animation-delay: var(--delay);  
  user-select: none;  
}
```

```
@keyframes float {  
  0%, 100% { transform: translateY(0) rotate(0); }  
  50% { transform: translateY(-20px) rotate(10deg); }  
}
```

```
.home-shell {  
  width: 100%;  
  max-width: 440px;  
  display: flex;  
  flex-direction: column;  
  gap: 20px;  
  position: relative;  
  z-index: 1;  
}
```

```
/* Logo 区域 */
```

```
.logo-area {  
  display: flex;
```

```
    justify-content: center;
    margin-bottom: 8px;
    opacity: 0;
    transform: scale(0.9);
    transition: all 0.6s var(--ease-out);
}

.is-ready .logo-area {
    opacity: 1;
    transform: scale(1);
}

.logo-circle {
    width: 84px;
    height: 84px;
    border-radius: 30px;
    background: white;
    display: flex;
    align-items: center;
    justify-content: center;
    position: relative;
    box-shadow: 0 8px 32px rgba(49, 120, 246, 0.12), inset 0 2px 4px white;
    border: 1px solid rgba(255, 255, 255, 0.8);
}

.logo-inner {
    display: flex;
    gap: 4px;
    font-size: 36px;
    font-weight: 900;
    line-height: 1;
}

.logo-plus { color: var(--candy-blue-dark); }
.logo-minus { color: var(--candy-mint-dark); }

.logo-sparkle {
    position: absolute;
    top: -12px;
    right: -12px;
    color: var(--candy-yellow-dark);
    filter: drop-shadow(0 0 8px rgba(245, 201, 74, 0.4));
    animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
    0%, 100% { transform: scale(1); opacity: 1; }
    50% { transform: scale(1.1); opacity: 0.8; }
}
```

```
/* 面板通用样式 */
.hero-panel,
.action-panel {
  background: rgba(255, 255, 255, 0.72);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.8);
  box-shadow: 0 12px 40px rgba(0, 0, 0, 0.04);
  opacity: 0;
  transform: translateY(16px);
  transition: opacity 0.6s var(--ease-out), transform 0.6s var(--ease-out);
}

.is-ready .hero-panel { transition-delay: 150ms; opacity: 1; transform: translateY(0); }
.is-ready .action-panel { transition-delay: 250ms; opacity: 1; transform: translateY(0); }

.hero-panel {
  border-radius: 32px;
  padding: 24px;
  text-align: center;
}

.hero-topline {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 12px;
  margin-bottom: 24px;
}

.mission-chip {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 6px 12px;
  border-radius: 999px;
  background: rgba(49, 120, 246, 0.06);
  color: var(--text-blue);
  font-size: 13px;
  font-weight: 700;
}

.mission-note {
  color: var(--text-secondary);
  font-size: 13px;
  font-weight: 600;
}

.hero-copy {
  margin-bottom: 28px;
}
```

```
}

.title {
  margin-bottom: 8px;
  color: var(--text-primary);
  font-size: 42px;
  font-weight: 800;
  letter-spacing: -0.02em;
}

.subtitle {
  color: var(--text-secondary);
  font-size: 16px;
  font-weight: 500;
}

/* 统计网格 */
.stats-grid {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 20px;
  padding: 16px;
  background: rgba(255, 255, 255, 0.5);
  border-radius: 20px;
  border: 1px solid rgba(255, 255, 255, 0.6);
}

.stat-item {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.stat-value {
  font-size: 24px;
  font-weight: 800;
  color: var(--text-primary);
  font-variant-numeric: tabular-nums;
}

.stat-label {
  font-size: 12px;
  font-weight: 600;
  color: var(--text-secondary);
}

.stat-divider {
  width: 1px;
```

```
    height: 32px;
    background: rgba(0, 0, 0, 0.06);
}
```

```
/* 首次用户欢迎提示 */
```

```
.welcome-hint {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    padding: 16px;
    background: rgba(255, 255, 255, 0.5);
    border-radius: 20px;
    border: 1px solid rgba(255, 255, 255, 0.6);
    color: var(--text-secondary);
    font-size: 14px;
    font-weight: 600;
}
```

```
.welcome-sparkle {
    color: var(--candy-yellow-dark);
    animation: pulse 2s ease-in-out infinite;
}
```

```
/* 操作区域 */
```

```
.action-panel {
    border-radius: 28px;
    padding: 12px;
    display: flex;
    flex-direction: column;
    gap: 12px;
}
```

```
.btn-main {
    width: 100%;
    height: 64px;
    border: none;
    border-radius: 22px;
    background: var(--candy-blue-dark);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 16px;
    font-size: 18px;
    font-weight: 800;
    box-shadow: 0 8px 24px rgba(49, 120, 246, 0.2);
    transition: all 0.3s var(--ease-out);
    cursor: pointer;
}
```

```
.btn-main-icon {
  width: 32px;
  height: 32px;
  background: rgba(255, 255, 255, 0.2);
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
}

.btn-text {
  letter-spacing: 0.02em;
}

.btn-arrow {
  transition: transform 0.3s var(--ease-out);
}

.btn-main:active {
  transform: scale(0.96);
  background: var(--brand-primary-dark);
}

.btn-main.is-leaving {
  opacity: 0.5;
  transform: scale(0.9) translateY(4px);
  filter: blur(2px);
}

@media (hover: hover) {
  .btn-main:hover {
    transform: translateY(-2px);
    box-shadow: 0 12px 32px rgba(49, 120, 246, 0.3);
  }

  .btn-main:hover .btn-arrow {
    transform: translateX(4px);
  }
}

@media (max-width: 420px) {
  .home-shell { gap: 16px; }
  .title { font-size: 36px; }
  .hero-panel { padding: 20px; }
  .stats-grid { gap: 12px; padding: 12px; }
  .stat-value { font-size: 20px; }
  .logo-circle { width: 72px; height: 72px; border-radius: 24px; }
  .logo-inner { font-size: 30px; }
}
```



```
</style>
```

```
// File: src/pages/NumberExplore.vue
```

```
<script setup>
```

```
import { computed, ref } from 'vue'
```

```
import { useRouter } from 'vue-router'
```

```
import { ArrowLeft, RotateCow, Target } from 'lucide-vue-next'
```

```
import NumberPad from '../components/NumberPad.vue'
```

```
import BallArray from '../components/BallArray.vue'
```

```
import { useSound } from '../composables/useSound'
```

```
const router = useRouter()
```

```
const { playClick, playSubmit } = useSound()
```

```
const QUICK_COUNTS = [10, 50, 100, 500, 1000]
```

```
const CHALLENGE_RANGES = [
```

```
  { label: '10 以内', min: 1, max: 10, hint: '先从少量小球开始，培养数量感。' },
  { label: '10 几', min: 11, max: 19, hint: '开始感受“十几”这个区间。' },
  { label: '20 几', min: 20, max: 29, hint: '先练习 20 多的数量，观察两排多一点的感觉。' },
  { label: '30 几', min: 30, max: 39, hint: '继续往上，看看 30 多和 20 多的差别。' },
  { label: '40 几', min: 40, max: 49, hint: '球阵会更满一些，试着先估整排。' },
  { label: '50 几', min: 50, max: 59, hint: '开始建立“半百左右”的数量感。' },
  { label: '60 几', min: 60, max: 69, hint: '先看大概几排，再判断剩下多少。' },
  { label: '70 几', min: 70, max: 79, hint: '数量越来越密了，先估整十会更稳。' },
  { label: '80 几', min: 80, max: 89, hint: '快接近 100 了，看看视觉上有多满。' },
  { label: '90 几', min: 90, max: 99, hint: '最后冲刺到 100 前，练习接近满格的感觉。' },
  { label: '100-120', min: 100, max: 120, hint: '刚过 100，先感受一层和两层之间的变化。' },
  { label: '121-150', min: 121, max: 150, hint: '继续练习 100 多，开始观察每层的大致分布。' },
  { label: '151-199', min: 151, max: 199, hint: '接近 200 之前，试着同时看层数和单层数量。' },
  { label: '200-300', min: 200, max: 300, hint: '进入几百个范围后，先估层数会更容易。' },
  { label: '301-500', min: 301, max: 500, hint: '球阵更厚了，适合练“整百 + 零头”的感觉。' },
  { label: '501-750', min: 501, max: 750, hint: '开始挑战大数量，先看大体体积再细估。' },
  { label: '751-1000', min: 751, max: 1000, hint: '最终高阶范围，练习接近满载的数量判断。' }
]
```

```
const mode = ref('explore')
```

```
const inputNumber = ref('')
```

```
const currentCount = ref(0)
```

```
const showResult = ref(false)
```

```
const isShaking = ref(false)
```

```
const statusMessage = ref('输入 1-1000，看看它会变成多少颗小球。')
```

```
const statusTone = ref('default')
```

```
const challengeTargetCount = ref(0)
```

```
const challengeRangeIndex = ref(0)
```

```
const challengeCorrectInRange = ref(0)
```

```
const challengeSolvedCount = ref(0)
```

```
const challengeLastGuess = ref(null)
```

```
const challengeLastCorrect = ref(false)
```

```
const isChallengeMode = computed(() => mode.value === 'challenge')
const currentChallengeRange = computed(() => CHALLENGE_RANGES[challengeRangeIndex.value])
const challengeProgressText = computed(() => `本档答对 ${challengeCorrectInRange.value} 题`)
const challengeStatusText = computed(() => {
  if (!isChallengeMode.value) return ''
  return `当前范围: ${currentChallengeRange.value.label} · ${challengeProgressText.value}`
})

const numberParts = computed(() => {
  const count = currentCount.value
  const hundreds = Math.floor(count / 100)
  const tens = Math.floor((count % 100) / 10)
  const ones = count % 10
  const parts = []

  if (hundreds > 0) parts.push(`${hundreds}个百`)
  if (tens > 0) parts.push(`${tens}个十`)
  if (ones > 0 || parts.length === 0) parts.push(`${ones}个一`)

  return parts.join(' + ')
})

function setStatus(message, tone = 'default') {
  statusMessage.value = message
  statusTone.value = tone
}

function pulseNumber() {
  isShaking.value = true
  window.setTimeout(() => {
    isShaking.value = false
  }, 280)
}

/**
 * Generates a random integer within a range.
 * @param {number} min – Minimum value.
 * @param {number} max – Maximum value.
 * @returns {number} Random integer.
 */
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

function prepareExploreMode() {
  inputNumber.value = ''
  currentCount.value = 0
  showResult.value = false
  challengeLastGuess.value = null
  setStatus('输入 1-1000, 看看它会变成多少颗小球。', 'default')
```

```
}

function prepareChallengeRound(keepInput = false) {
  const range = currentChallengeRange.value

  challengeTargetCount.value = randomInt(range.min, range.max)
  showResult.value = false
  currentCount.value = 0
  challengeLastGuess.value = null
  challengeLastCorrect.value = false

  if (!keepInput) {
    inputNumber.value = ''
  }

  setStatus(`挑战范围: ${range.label}。${range.hint}`, 'default')
}

function startChallenge(resetProgress = false) {
  if (resetProgress) {
    challengeRangeIndex.value = 0
    challengeCorrectInRange.value = 0
    challengeSolvedCount.value = 0
  }

  prepareChallengeRound()
}

function selectChallengeRange(index) {
  if (challengeRangeIndex.value === index && !showResult.value) return

  playClick()
  challengeRangeIndex.value = index
  challengeCorrectInRange.value = 0
  prepareChallengeRound()
}

function switchMode(nextMode) {
  if (mode.value === nextMode) return

  playClick()
  mode.value = nextMode
  isShaking.value = false

  if (nextMode === 'challenge') {
    startChallenge(true)
    return
  }

  prepareExploreMode()
}
```

```
}

function handleInput(num) {
  playClick()

  if (inputNumber.value === '' && num === 0) {
    setStatus('从 1 开始输入，0 不能单独展示。', 'warning')
    pulseNumber()
    return
  }

  const nextValue = `${inputNumber.value}${num}`
  const parsed = Number.parseInt(nextValue, 10)

  if (Number.isNaN(parsed) || parsed > 1000) {
    setStatus('最大可以输入到 1000。', 'warning')
    pulseNumber()
    return
  }

  inputNumber.value = nextValue

  if (isChallengeMode.value) {
    setStatus(`你猜的是 ${parsed}。${challengeStatusText.value}`, 'default')
    return
  }

  setStatus(`准备探索 ${parsed}。`, 'default')
}

function handleDelete() {
  playClick()
  inputNumber.value = inputNumber.value.slice(0, -1)

  if (isChallengeMode.value) {
    setStatus(inputNumber.value ? `你猜的是 ${inputNumber.value}。${challengeStatusText.value}` : `${challengeStatusText.value}, 先看球阵再猜数字。`, 'default')
    return
  }

  setStatus(inputNumber.value ? `当前输入 ${inputNumber.value}。` : `输入 1-1000, 看看它会变成多少颗小球。`, 'default')
}

function handleExploreSubmit(num) {
  playSubmit()
  currentCount.value = num
  showResult.value = true
  setStatus(`正在探索 ${num}。`, 'success')
}

function handleChallengeSubmit(num) {
```

```
const target = challengeTargetCount.value
const correct = num === target

challengeLastGuess.value = num
challengeLastCorrect.value = correct
currentCount.value = target
showResult.value = true

if (correct) {
  playSubmit()
  challengeSolvedCount.value += 1
  challengeCorrectInRange.value += 1
  setStatus(`猜对了。答案就是 ${target}。`, 'success')
  return
}

playClick()
setStatus(`这次没猜中，正确答案是 ${target}。`, 'warning')
}

function handleSubmit() {
  const num = Number.parseInt(inputNumber.value, 10)

  if (Number.isNaN(num) || num < 1 || num > 1000) {
    playClick()
    setStatus(`请输入 1 到 1000 之间的数字。`, 'warning')
    pulseNumber()
    return
  }

  if (isChallengeMode.value) {
    handleChallengeSubmit(num)
    return
  }

  handleExploreSubmit(num)
}

function applyQuickCount(num) {
  playClick()
  inputNumber.value = String(num)
  setStatus(`已选择 ${num}，点击确认就能看结果。`, 'default')
}

function updateCurrentCount(nextCount) {
  const safeCount = Math.min(1000, Math.max(1, nextCount))

  if (safeCount === currentCount.value) {
    setStatus(`已经到 ${safeCount} 了。`, 'warning')
    pulseNumber()
  }
}
```

```
        return
    }

    playClick()
    currentCount.value = safeCount
    inputNumber.value = String(safeCount)
    setStatus(`正在探索 ${safeCount}。`, 'success')
}

function stepCount(step) {
    updateCurrentCount(currentCount.value + step)
}

function jumpToCount(num) {
    updateCurrentCount(num)
}

function resetExplore() {
    playClick()

    if (isChallengeMode.value) {
        startChallenge(false)
        return
    }

    prepareExploreMode()
}

function nextChallenge() {
    playClick()
    prepareChallengeRound()
}

function retryChallengeRange() {
    playClick()
    challengeCorrectInRange.value = 0
    prepareChallengeRound()
}

function goHome() {
    playClick()
    router.push('/')
}
</script>

<template>
<div class="explore-page">
    <Transition name="fade" mode="out-in">
        <div v-if="!showResult" class="view-wrapper">
            <header class="top-bar">
```

```

<button class="back-btn" aria-label="返回首页" @click="goHome">
  <ArrowLeft :size="22" stroke-width="2.8" />
</button>
</header>

<main class="input-screen">
  <section class="mode-switch" aria-label="模式切换">
    <button
      class="mode-btn"
      :class="{ 'is-active': mode === 'explore' }"
      type="button"
      @click="switchMode('explore')"
    >
      自由探索
    </button>
    <button
      class="mode-btn"
      :class="{ 'is-active': mode === 'challenge' }"
      type="button"
      @click="switchMode('challenge')"
    >
      挑战模式
    </button>
  </section>

  <section v-if="isChallengeMode" class="range-selector-shell">
    <div class="range-selector-header">
      <div class="counter-badge range-title-badge">选择训练范围</div>
      <p class="range-selector-copy">先选范围，再看球阵猜数字。</p>
    </div>

    <div class="range-selector" aria-label="挑战范围选择">
      <button
        v-for="(range, index) in CHALLENGE_RANGES"
        :key="range.label"
        class="range-chip"
        :class="{ 'is-active': challengeRangeIndex === index }"
        type="button"
        @click="selectChallengeRange(index)"
      >
        {{ range.label }}
      </button>
    </div>
  </section>

  <section
    :class="isChallengeMode ? 'challenge-workspace' : 'explore-workspace'"
    :aria-label="isChallengeMode ? '挑战练习区' : '自由探索区'"
  >
    <section class="number-stage" :class="{ 'is-shaking': isShaking }">

```

```
<div class="number-card">
  <div class="number-card-top">
    <div class="counter-badge">{{ isChallengeMode ? '数字挑战' : '数字探索' }}</div>
    <div v-if="isChallengeMode" class="counter-badge range-badge">{{ currentChallengeRange.label }}</div>
  </div>
  <div class="number-display font-number">
    <div class="big-number" :class="{ 'is-empty': !inputNumber }">
      {{ inputNumber || '?' }}
    </div>
  </div>
  <p v-if="!isChallengeMode" class="helper-copy">
    输入一个数字，马上看看它会变成多少颗球。
  </p>
  <p class="status-copy" :class="{ 'is-': statusTone }" aria-live="polite">
    {{ isChallengeMode ? challengeStatusText : statusMessage }}
  </p>

  <div v-if="isChallengeMode" class="challenge-meta">
    <div class="challenge-card">
      <span class="challenge-label">范围</span>
      <strong>{{ currentChallengeRange.label }}</strong>
    </div>
    <div class="challenge-card">
      <span class="challenge-label">本档</span>
      <strong>{{ challengeCorrectInRange }}题</strong>
    </div>
    <div class="challenge-card">
      <span class="challenge-label">累计</span>
      <strong>{{ challengeSolvedCount }}</strong>
    </div>
  </div>

  <div v-else class="quick-counts" aria-label="快捷数字">
    <button
      v-for="num in QUICK_COUNTS"
      :key="num"
      class="quick-chip"
      type="button"
      @click="applyQuickCount(num)"
    >
      {{ num }}
    </button>
  </div>
</div>
</section>

<section v-if="isChallengeMode" class="challenge-preview-shell">
  <div class="preview-copy">
    <Target :size="18" />
    <span>看球阵猜数字</span>
  </div>
</section>
```



```

    </div>
    <div class="challenge-ball-shell">
        <BallArray :count="challengeTargetCount" size="compact" />
    </div>
</section>

<section class="pad-stage">
    <NumberPad size="compact" @input="handleInput" @delete="handleDelete" @submit="handleSubmit" />
</section>
</main>
</div>

<main v-else class="result-screen view-wrapper">
    <section class="result-number-shell">
        <div class="result-number-block">
            <div class="counter-badge result-badge">
                {{ isChallengeMode ? (challengeLastCorrect ? '挑战成功' : '答案公布') : '探索结果' }}
            </div>
            <div class="big-number font-number">{{ currentCount }}</div>
            <p v-if="isChallengeMode" class="result-copy">
                {{ challengeLastCorrect ? `你猜对了，答案就是 ${currentCount}。` : `你猜 ${challengeLastGuess}，正确答案是 ${currentCount}。` }}
            </p>
            <p v-else class="result-copy">{{ currentCount }} = {{ numberParts }}</p>
        </div>
    </section>

    <section class="result-ball-shell">
        <BallArray :count="currentCount" />
    </section>

    <section v-if="isChallengeMode" class="result-tools" aria-label="挑战结果操作">
        <div class="challenge-summary">
            <div class="challenge-card">
                <span class="challenge-label">当前范围</span>
                <strong>{{ currentChallengeRange.label }}</strong>
            </div>
            <div class="challenge-card">
                <span class="challenge-label">已答对</span>
                <strong>{{ challengeSolvedCount }}</strong>
            </div>
            <div class="challenge-card">
                <span class="challenge-label">本档训练</span>
                <strong>{{ challengeCorrectInRange }}题</strong>
            </div>
        </div>

        <button class="play-again-btn" @click="nextChallenge">
            <Target :size="20" />
            <span>下一题</span>
        </button>
    </section>

```

```

</button>

<button class="secondary-btn" type="button" @click="retryChallengeRange">
  重新练这一档
</button>
</section>

<section v-else class="result-tools" aria-label="连续探索操作">
  <div class="step-controls">
    <button class="step-btn" type="button" @click="stepCount(-1)">-1</button>
    <button class="step-btn" type="button" @click="stepCount(1)">+1</button>
    <button class="step-btn" type="button" @click="stepCount(10)">+10</button>
  </div>

  <div class="quick-counts result-counts" aria-label="常用数字">
    <button
      v-for="num in QUICK_COUNTS"
      :key="`result-${num}`"
      class="quick-chip"
      type="button"
      @click="jumpToCount(num)"
    >
      {{ num }}
    </button>
  </div>

  <button class="play-again-btn" @click="resetExplore">
    <RotateCcw :size="20" />
    <span>重新输入</span>
  </button>
</section>
</main>
</Transition>
</div>
</template>

<style scoped>
.explore-page {
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  padding-top: env(safe-area-inset-top);
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  background:
    radial-gradient(circle at top, rgba(92, 157, 255, 0.12) 0%, rgba(92, 157, 255, 0) 38%),
    linear-gradient(180deg, #f8fbff 0%, #edf4ff 100%);
  overflow: hidden;
}

.top-bar {

```

```
display: flex;
align-items: center;
padding: 12px 16px 4px;
}

.back-btn {
width: 48px;
height: 48px;
border: 1px solid rgba(92, 157, 255, 0.12);
border-radius: 18px;
background: rgba(255, 255, 255, 0.88);
box-shadow: 0 12px 28px rgba(49, 120, 246, 0.12);
display: inline-flex;
align-items: center;
justify-content: center;
color: var(--brand-primary);
backdrop-filter: blur(12px);
-webkit-backdrop-filter: blur(12px);
transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);
}

.back-btn:focus-visible,
.play-again-btn:focus-visible,
.quick-chip:focus-visible,
.step-btn:focus-visible,
.mode-btn:focus-visible,
.secondary-btn:focus-visible,
.range-chip:focus-visible {
outline: none;
box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}

.back-btn:active,
.play-again-btn:active {
transform: scale(0.95);
}

.view-wrapper {
flex: 1;
display: flex;
flex-direction: column;
}

.input-screen,
.result-screen {
flex: 1;
min-height: 0;
display: flex;
flex-direction: column;
padding: 8px 14px 0;
```

```
}
```

```
.input-screen {  
  gap: 12px;  
}
```

```
.result-screen {  
  gap: 12px;  
  padding-inline: 10px;  
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);  
  overflow-y: auto;  
  -webkit-overflow-scrolling: touch;  
}
```

```
.mode-switch {  
  display: grid;  
  grid-template-columns: repeat(2, minmax(0, 1fr));  
  gap: 10px;  
}
```

```
.mode-btn,  
.secondary-btn {  
  min-height: 48px;  
  border: 1px solid rgba(92, 157, 255, 0.14);  
  border-radius: 18px;  
  background: rgba(255, 255, 255, 0.84);  
  color: var(--text-blue);  
  font-size: 15px;  
  font-weight: 800;  
  transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard);  
}
```

```
.mode-btn.is-active {  
  background: linear-gradient(180deg, #6da8ff 0%, #4b86f3 100%);  
  border-color: rgba(75, 134, 243, 0.24);  
  color: #fff;  
  box-shadow: 0 12px 22px rgba(75, 134, 243, 0.18);  
}
```

```
.range-selector-shell,  
.challenge-workspace {  
  border: 1px solid rgba(92, 157, 255, 0.12);  
  border-radius: 30px;  
  background: rgba(255, 255, 255, 0.9);  
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);  
  backdrop-filter: blur(14px);  
  -webkit-backdrop-filter: blur(14px);  
}
```

```
.range-selector-shell {
```

```
padding: 10px 12px;
}
```

```
.range-selector-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 8px;
}
```

```
.range-title-badge {
  flex-shrink: 0;
}
```

```
.range-selector-copy {
  margin: 0;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  text-align: right;
}
```

```
.challenge-workspace {
  padding: 8px;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

```
.explore-workspace {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

```
.number-stage,
.result-number-shell,
.result-ball-shell,
.play-again-btn,
.step-btn,
.challenge-preview-shell {
  position: relative;
  border: 1px solid rgba(92, 157, 255, 0.12);
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
  backdrop-filter: blur(14px);
  -webkit-backdrop-filter: blur(14px);
}
```

```
.number-stage,  
.result-number-shell,  
.challenge-preview-shell {  
  overflow: hidden;  
  border-radius: 34px;  
}
```

```
.number-stage {  
  padding: 10px 12px 12px;  
  background: rgba(255, 255, 255, 0.96);  
  border: 1px solid rgba(92, 157, 255, 0.14);  
  box-shadow: 0 20px 36px rgba(58, 87, 152, 0.1);  
}
```

```
.number-card {  
  width: 100%;  
}
```

```
.number-card-top,  
.number-display {  
  display: flex;  
  align-items: center;  
}
```

```
.number-card-top {  
  justify-content: flex-start;  
  gap: 8px;  
  margin-bottom: 8px;  
}
```

```
.counter-badge {  
  padding: 8px 12px;  
  border-radius: var(--radius-full);  
  background: rgba(92, 157, 255, 0.1);  
  border: 1px solid rgba(92, 157, 255, 0.12);  
  color: var(--brand-primary);  
  font-size: var(--font-sm);  
  font-weight: 800;  
}
```

```
.range-badge {  
  background: rgba(107, 203, 119, 0.12);  
  border-color: rgba(107, 203, 119, 0.18);  
  color: var(--text-teal);  
}
```

```
.number-display {  
  justify-content: center;  
  min-height: 76px;  
}
```

```
.big-number {
  position: relative;
  z-index: 1;
  min-width: clamp(88px, 24vw, 132px);
  padding: 12px 14px;
  border-radius: var(--radius-lg);
  border: 2px solid rgba(92, 157, 255, 0.18);
  background: linear-gradient(180deg, #ffffff 0%, #f1f7ff 100%);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.92), 0 10px 20px rgba(92, 157, 255, 0.08);
  color: var(--text-blue-dark);
  font-size: clamp(48px, 14vw, 76px);
  font-weight: 800;
  line-height: 1;
  text-align: center;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);
}

.big-number.is-empty {
  color: var(--text-muted);
}

.helper-copy,
.status-copy,
.result-copy {
  margin: 0;
  text-align: center;
}

.helper-copy {
  margin-top: 8px;
  color: var(--text-secondary);
  font-size: 14px;
  line-height: 1.5;
}

.status-copy {
  min-height: 16px;
  margin-top: 6px;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  line-height: 1.25;
}

.status-copy.is-warning {
  color: var(--text-warning);
}

.status-copy.is-success {
```

```
    color: var(--text-success);
}

.challenge-meta,
.challenge-summary {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 6px;
    margin-top: 8px;
}

.range-selector {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 6px;
    margin-top: 2px;
}

.range-chip {
    min-height: 44px;
    padding: 0 12px;
    border: 1px solid rgba(92, 157, 255, 0.16);
    border-radius: 999px;
    background: rgba(255, 255, 255, 0.9);
    color: var(--text-blue);
    font-size: 13px;
    font-weight: 800;
    transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), color var(--duration-fast) var(--ease-standard);
}

.range-chip.is-active {
    background: linear-gradient(180deg, #6ecb96 0%, #4aac70 100%);
    border-color: rgba(74, 172, 112, 0.22);
    color: #fff;
    box-shadow: 0 10px 18px rgba(74, 172, 112, 0.16);
}

.challenge-card {
    min-height: 44px;
    padding: 6px 4px;
    border: 1px solid rgba(92, 157, 255, 0.12);
    border-radius: 14px;
    background: rgba(255, 255, 255, 0.82);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    gap: 2px;
    text-align: center;
```



```
}
```

```
.challenge-label {  
  color: var(--text-blue-light);  
  font-size: 10px;  
  font-weight: 700;  
}
```

```
.challenge-card strong {  
  color: var(--text-blue-dark);  
  font-size: 13px;  
  font-weight: 900;  
}
```

```
.quick-counts {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: center;  
  gap: 8px;  
  margin-top: 14px;  
}
```

```
.quick-chip {  
  min-height: 44px;  
  padding: 0 14px;  
  border: 1px solid rgba(92, 157, 255, 0.16);  
  border-radius: 999px;  
  background: rgba(255, 255, 255, 0.9);  
  color: var(--brand-primary);  
  font-size: 14px;  
  font-weight: 800;  
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);  
}
```

```
.quick-chip:active,  
.step-btn:active,  
.mode-btn:active,  
.secondary-btn:active,  
.range-chip:active {  
  transform: scale(0.96);  
}
```

```
.number-stage.is-shaking .big-number {  
  border-color: rgba(255, 107, 107, 0.4);  
  animation: toy-shake 0.28s ease;  
}
```

```
.number-stage.is-shaking {  
  animation: none;  
}
```

```
.challenge-preview-shell {  
  padding: 8px;  
  border-radius: 22px;  
  background: linear-gradient(180deg, rgba(247, 250, 255, 0.98) 0%, rgba(235, 243, 255, 0.95) 100%);  
}
```

```
.preview-copy {  
  display: inline-flex;  
  align-items: center;  
  gap: 8px;  
  color: var(--text-blue);  
  font-size: 12px;  
  font-weight: 800;  
  margin-bottom: 6px;  
}
```

```
.challenge-ball-shell {  
  border-radius: 20px;  
  overflow: hidden;  
}
```

```
.result-number-shell {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 18px 18px 16px;  
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(241, 247, 255, 0.96) 100%);  
}
```

```
.result-number-block {  
  width: 100%;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  gap: 10px;  
}
```

```
.result-badge {  
  background: rgba(107, 203, 119, 0.12);  
  border-color: rgba(107, 203, 119, 0.18);  
  color: var(--text-teal);  
}
```

```
.result-copy {  
  color: var(--text-blue-light);  
  font-size: 15px;  
  font-weight: 800;  
  line-height: 1.5;  
}
```

```
.result-ball-shell {  
  width: 100%;  
  max-width: min(100%, calc(100dvh - 320px));  
  aspect-ratio: 1 / 1;  
  margin: 0 auto;  
  border-radius: 34px;  
  padding: 4px;  
  overflow: hidden;  
}
```

```
.result-tools {  
  display: flex;  
  flex-direction: column;  
  gap: 12px;  
}
```

```
.step-controls {  
  display: grid;  
  grid-template-columns: repeat(3, minmax(0, 1fr));  
  gap: 10px;  
}
```

```
.step-btn {  
  min-height: 52px;  
  border-radius: 20px;  
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(242, 247, 255, 0.96) 100%);  
  color: var(--text-blue-dark);  
  font-size: 18px;  
  font-weight: 900;  
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);  
}
```

```
.result-counts {  
  margin-top: 0;  
}
```

```
.play-again-btn {  
  width: 100%;  
  min-height: 64px;  
  border-radius: 24px;  
  padding: 0 20px;  
  display: inline-flex;  
  align-items: center;  
  justify-content: center;  
  gap: 10px;  
  color: #fff;  
  font-size: 22px;  
  font-weight: 900;  
  background: linear-gradient(180deg, #62c98a 0%, #48aa70 100%);  
}
```

```
border-color: rgba(74, 172, 112, 0.24);
box-shadow: 0 12px 26px rgba(74, 172, 112, 0.2);
}

.secondary-btn {
  color: var(--text-blue-light);
}

@media (hover: hover) {
  .quick-chip:hover,
  .step-btn:hover,
  .secondary-btn:hover,
  .range-chip:hover:not(.is-active) {
    border-color: rgba(92, 157, 255, 0.28);
    background: var(--bg-light);
  }

  .mode-btn:hover:not(.is-active) {
    background: rgba(255, 255, 255, 0.94);
  }

  .play-again-btn:hover {
    box-shadow: 0 16px 30px rgba(74, 172, 112, 0.24);
  }
}

.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.35s ease, transform 0.35s ease;
}

.fade-enter-from,
.fade-leave-to {
  opacity: 0;
  transform: scale(0.96) translateY(20px);
}

@media (max-width: 420px) {
  .range-selector-header {
    flex-direction: column;
    align-items: flex-start;
  }

  .range-selector-copy {
    text-align: left;
  }

  .number-stage {
    padding: 8px 10px 10px;
  }
}
```

```
.number-card-top {
  margin-bottom: 8px;
}

.number-display {
  min-height: 66px;
}

.big-number {
  min-width: 72px;
  padding: 8px 10px;
}

.challenge-meta,
.challenge-summary {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

.challenge-workspace {
  padding: 6px;
  gap: 6px;
}

.result-copy {
  font-size: 14px;
}

.step-controls {
  gap: 8px;
}

.step-btn {
  min-height: 48px;
  border-radius: 18px;
  font-size: 17px;
}
}

@media (max-width: 360px) {
  .counter-badge {
    padding: 8px 10px;
    font-size: 12px;
  }

  .number-display {
    min-height: 92px;
  }

  .big-number {
```

```
        font-size: 42px;
    }
}

@media (max-width: 959px) and (max-height: 860px) {
    .number-stage {
        padding-top: 8px;
        padding-bottom: 8px;
    }

    .number-display {
        min-height: 88px;
    }

    .result-ball-shell {
        max-width: min(100%, calc(100dvh - 300px));
    }
}

@media (min-width: 768px) {
    .number-stage {
        padding: 14px 18px 18px;
    }
}
</style>
```